

ASTERIA

AI-Based Virtual Camera Tracking
for **Free-Space Optical Communication**

Technical Report

Team Name - 8Bit

Team ID - 123585

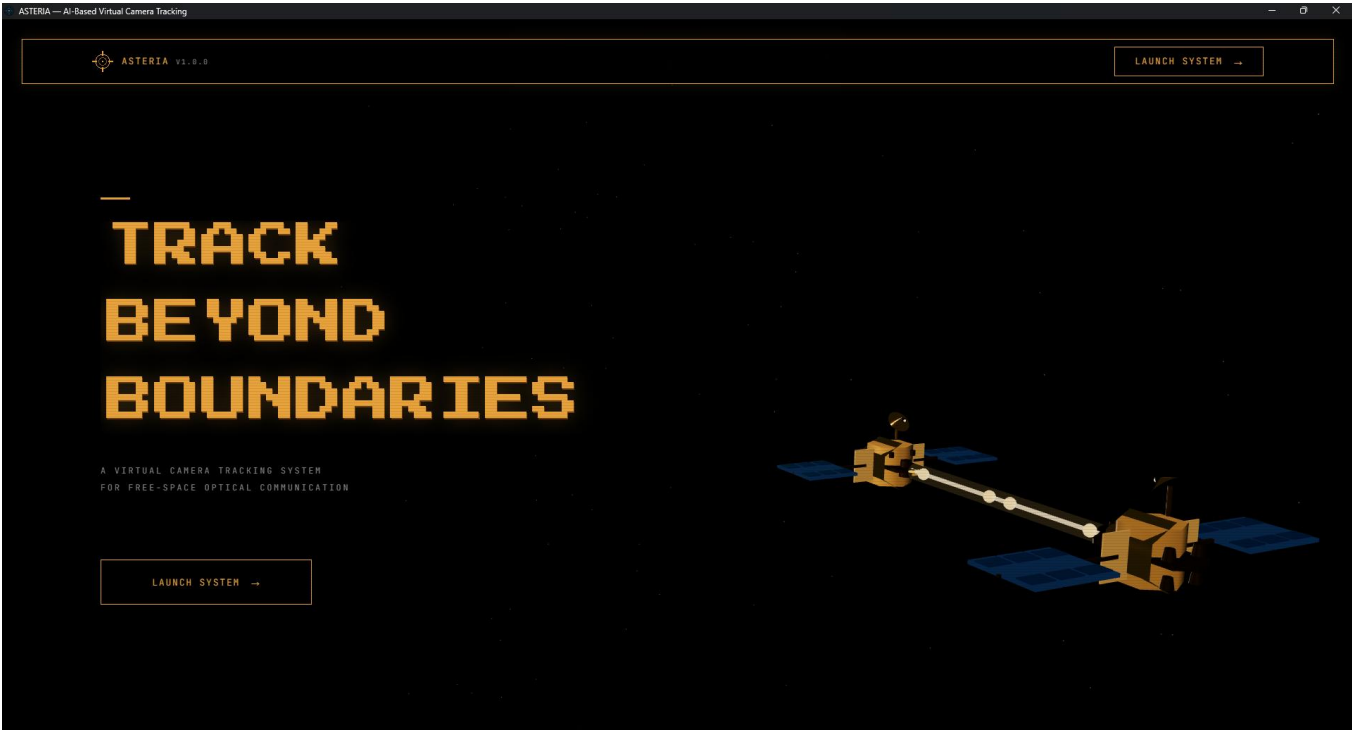


Figure 1. ASTERIA mission-control landing interface.

1. Problem Understanding

FSOC Pointing, Acquisition and Tracking

Free-space optical communication requires accurate line-of-sight alignment because the optical beam is narrow. ASTERIA addresses the software **PAT** problem: detect a beacon, estimate its **image-space** state, steer a virtual camera, maintain lock, and recover when the target is lost.

From a control-system perspective, the problem is a coupled perception-estimation-actuation loop. Detection quality determines the quality of the measurement; estimation reduces the effect of measurement noise and temporary dropout; the controller converts residual error into constrained actuator motion. The simulator makes each of these stages observable so that a failure can be traced to perception, state estimation, control authority, search coverage, or environmental degradation rather than appearing only as a final tracking failure.

- Acquire a target within the camera field of view.
- Estimate beacon position and short-term motion.
- Convert pixel error into **pan/tilt** correction.
- Maintain tracking under noise, target motion and camera disturbances.
- Detect loss and execute bounded **reacquisition**.
- Measure acquisition, error, **lock retention**, FPS and latency.

Configured requirement	Target
Camera / FOV	640×480; 4° horizontal; 3° vertical
Nominal rate	30 FPS
Acquisition	≤ 2.0 s
Tracking error	≤ 10 px
Target loss	< 5%
Reacquisition	≤ 1.0 s
Processing	≥ 20 FPS

Scope boundary: this is a software simulator and algorithm-validation platform. It does not by itself represent a calibrated physical gimbal, flight computer, optical terminal, or field safety system.

PS169 baseline	Target	Implementation evidence
Acquisition time	≤ 2 s	RunMetrics / acceptance tests
Tracking error	≤ 10 px	Lock threshold / PS169 checks
Target loss rate	< 5 %	Lock-retention diagnostics
Reacquisition time	≤ 1 s	Loss/reacquisition logic
Frame rate	≥ 20 FPS; 30 Hz target	Camera/simulation loop

The separation between simulation truth and observed image data is important for evaluation: the simulator may retain **ground-truth** state for scoring, while the detector receives only the rendered and degraded image. This enables controlled experiments without turning the tracking loop into a **ground-truth** shortcut.

2. System Architecture

Application layers and communication paths

The main branch uses a service-oriented, three-layer application structure. The Python/FastAPI service owns the **PAT** simulation, image formation, detection, **Kalman** estimation, **PID** control, metrics, **SQLite** persistence and report generation. The React/TypeScript frontend provides mission controls, live telemetry, 2-D/3-D visualization, analytics and report access. A separate Node/Express service exposes the **Gemini** engineering-copilot route. The Electron layer packages these services into a desktop workflow and supervises their local processes.

This separation provides a clear boundary for real-time behavior. The hard tracking loop remains in Python, where the frame sequence is causally ordered: target update, camera projection, frame rendering, detection, **Kalman** update/prediction, error calculation, **PID** control, and finally metrics/telemetry. The frontend consumes this state asynchronously through **WebSocket** updates, while configuration and lifecycle operations use **REST**. Database writes are kept off the event loop, preventing persistence work from directly blocking the control calculation.

Layer	Technology	Primary responsibility
Frontend	React 19 + TypeScript + Vite	Mission UI, telemetry, controls, charts and reports
PAT engine	Python + FastAPI	Simulation, vision, Kalman, PID, state machine and metrics
Copilot	Node + Express + TypeScript	Gemini-based engineering assistance
Persistence	SQLite / WAL	Runs, scenarios and telemetry

Live processing path

Target / video → image detector → **Kalman filter** → angular error → **PID** → camera/control state → telemetry → UI

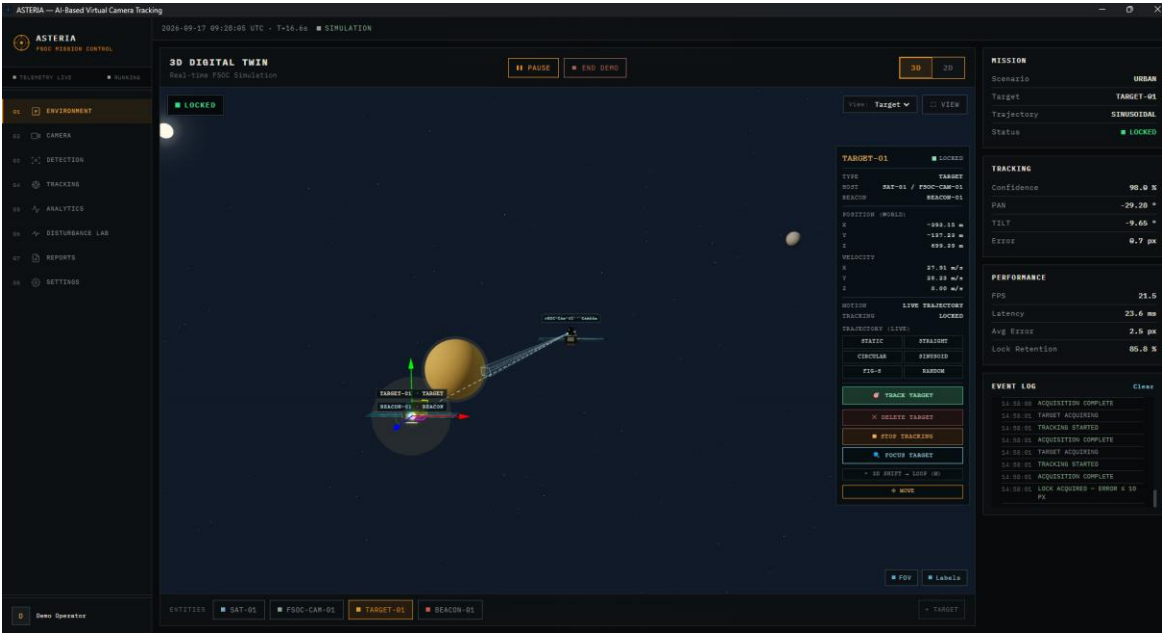


Figure 2. 3D digital-twin view showing target, beacon, camera and tracking state.

REST is used for configuration/control operations; **WebSocket** is used for live telemetry and video-processing updates. The frontend keeps a bounded history to prevent unbounded client-side memory growth.

This division also reduces coupling between operator commands and the high-rate telemetry stream. Configuration requests can be handled transactionally, while the **WebSocket** channel continuously publishes the current **tracking state** required by the dashboard.

3. Software Modules

Core implementation modules

Layer	Main implementation	Primary responsibility
Simulation	Target, camera, disturbances	World state, trajectories, motion perturbations
Vision	Frame renderer + ImageBeaconDetector	Degraded images, candidates, confidence
Prediction	KalmanFilter2D	Position/velocity prediction and uncertainty
Control	PIDController	Bounded pan/tilt corrections
Orchestration	FastAPI + SimulationEngine	Lifecycle, REST, WebSocket and loop ordering
Persistence	SQLite + reports	Runs, telemetry, CSV/json/PDF
Interface	React/TypeScript + Electron	Visualization, operator controls, desktop packaging

Module	Technical function
simulation/engine.py	Orchestrates target, camera, detector, Kalman, PID, disturbances and metrics.
simulation/camera.py	3-D camera pose, platform motion, FOV and world-to-pixel projection.
simulation/target.py	Target state, beacon offset and motion trajectories.
simulation/disturbances.py	Atmospheric, vibration, sensor and target-motion disturbances.
vision/detector.py	Adaptive image detection, candidate scoring and temporal continuity.
vision/video_processor.py	MP4 decoding and frame-by-frame tracking pipeline.
prediction/kalman.py	2-D constant-velocity state estimation.
control/pid.py	Dual-axis PID with anti-windup, filtering and rate limits.
analytics/metrics.py	Error, RMSE, acquisition, loss, retention, FPS and latency.
database/models.py	SQLite scenario/run/telemetry persistence.

Camera and geometry

The camera model converts the target's world-space position into camera-relative coordinates using the current **pan/tilt** orientation. A target behind the aperture or outside the horizontal/vertical field of view is treated as unavailable to the camera. Otherwise, the horizontal and vertical angular offsets are converted to image coordinates. The reviewed runtime uses a 640×480 image, 4° horizontal FOV, 3° vertical FOV and 30 Hz frame rate, giving a nominal 160 pixels/degree scale in each image axis. Pan and tilt are bounded so the virtual gimbal behaves like a constrained actuator rather than an instantaneous look-at camera.

This geometry is central to controller correctness because an incorrect sign or scale would cause the gimbal to amplify rather than reduce pointing error. The repository therefore keeps the image-origin convention, FOV constants and degree/radian conversions in a shared configuration boundary instead of reproducing them separately across modules.

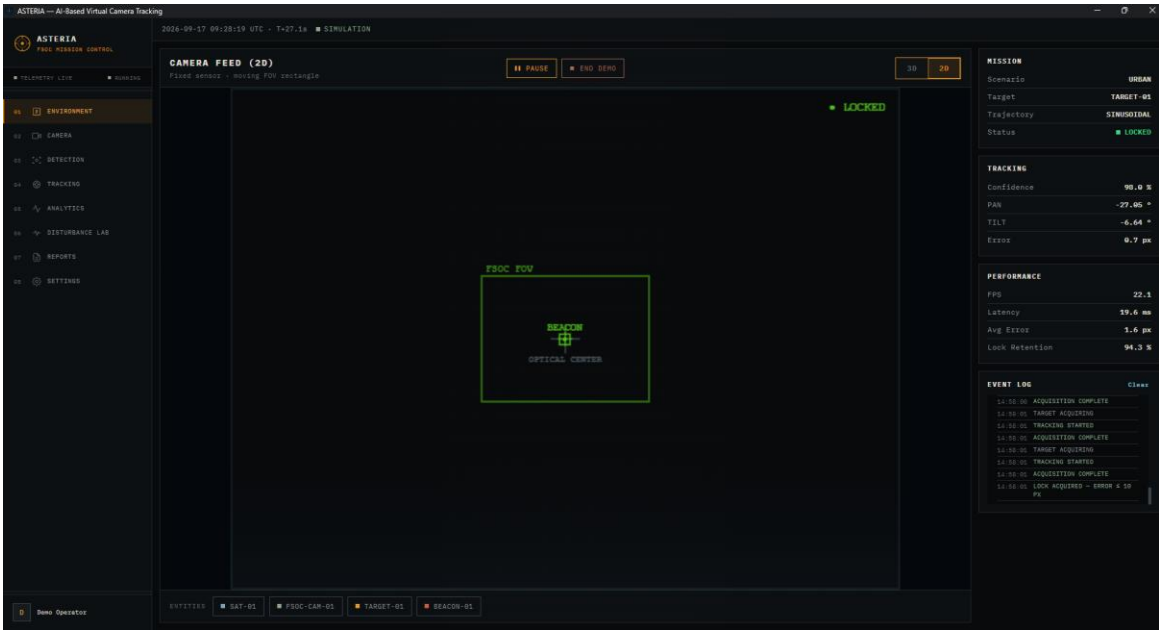


Figure 3. 2-D camera-feed view showing the optical center, FOV and locked beacon.

4. Tracking Methods

Detection, estimation and control

4.1 Beacon detection

The active detector is a robust, candidate-based classical computer-vision pipeline. It first applies a median filter to suppress isolated salt-and-pepper specks, then computes frame statistics and selects an adaptive high-intensity threshold. Connected components are generated and filtered by area, span and border-artifact rules. Remaining candidates are scored using peak and mean brightness, local contrast, aspect ratio, fill ratio, expected beacon area and temporal/predicted-position consistency. Long glare components receive a second high-threshold pass to recover a compact bright core. The selected candidate is returned with centroid, bounding box, confidence, inference time and candidate diagnostics.

The candidate-scoring design is deliberately interpretable. Candidate boxes, centroids, scores and confidence values remain available for UI diagnostics, allowing false locks to be traced to brightness, contrast, shape, size or association behavior. A nearest-neighbor association radius discourages sudden jumps to unrelated bright blobs, while a global fallback still allows genuine target re-entry. This makes the detector suitable for a research simulator where failure analysis and repeatability are as important as raw detection accuracy.

4.2 Kalman state estimation

KalmanFilter2D uses the state vector $x = [x, y, vx, vy]^T$ and a centroid measurement $[x, y]$. The prediction step advances position using the measured frame interval and the constant-velocity model; the update step incorporates the detector measurement through the innovation covariance and **Kalman** gain. The implementation exposes predicted position, velocity, covariance information and convergence/reset state through telemetry. When the detector temporarily misses the beacon, the tracker can coast on the predicted state for a bounded interval instead of immediately treating the target as permanently lost.

The state estimator therefore serves two purposes: smoothing noisy centroid measurements and providing a physically plausible short-term prediction. Its uncertainty value is also exported to telemetry, giving the operator a direct indication of how much confidence the estimator has in its current state.

4.3 PID control

The dual-axis **PID** controller independently computes pan and tilt corrections from the pointing error. The proportional term responds to the current error, the integral term removes persistent bias, and the derivative term reacts to error change. Integral accumulation is bounded to reduce windup during actuator saturation, while the derivative contribution is filtered to limit amplification of noisy measurements. The resulting command is limited to the configured angular rate and integrated over dt before the camera applies it, producing physically interpretable slew, settling, overshoot and disturbance-rejection behavior.

Because the controller commands angular rate rather than teleporting the camera, the simulation exposes practical control characteristics such as convergence time, overshoot, saturation, steady-state error and disturbance rejection. The dt -based integration also makes the controller less dependent on an assumed frame count, which is important when comparing runs with different processing loads.

Control law: $u(t) = K_p \cdot e(t) + K_i \cdot \int e(t) dt + K_d \cdot de(t)/dt$

The controller is intentionally rate-limited, so the camera responds through incremental motion instead of instantly jumping to the required angle. This makes transient response, saturation, settling and disturbance rejection observable in the simulation.



Figure 4. Camera-control panel with pan/tilt commands, PID configuration and control-error plots.

5. Tracking State Machine and Recovery

Maintaining lock under target loss

The tracking lifecycle separates acquisition from stable lock. The implementation uses consecutive low-error frames before locked and hysteresis before unlocking, reducing state flicker from a single noisy measurement.

Operationally, the state sequence distinguishes ready, searching, detected, acquiring, tracking, locked, lost, reacquiring and error. locked is based on consecutive low-error frames rather than one favorable measurement, and hysteresis uses a larger error boundary before unlocking. During short detector gaps, the **Kalman** estimate can bridge the missing observations; after the configured grace period, the tracker enters loss handling and uses a bounded search/**reacquisition** strategy. This avoids both premature unlocks and indefinite reliance on stale predictions.

State	Purpose
ready	System initialized and waiting for a run.
searching	Camera performs bounded search for a beacon.
detected	Valid candidate has been identified.
acquiring	Controller converges toward the target.
tracking	Closed-loop tracking is active.
locked	Residual error is inside the lock condition.
lost	Measurement unavailable beyond the grace period.
reacquiring	Search/recovery behavior attempts to restore lock.

During temporary dropout, the **Kalman** state can coast for roughly one second. Longer loss transitions the tracker toward lost/**reacquisition**. Search is bounded rather than teleporting the camera directly to a known target coordinate.

This recovery strategy is significant because a short detector miss does not immediately destroy the track. Prediction provides continuity, while the explicit state machine prevents indefinite use of an unconfirmed prediction.

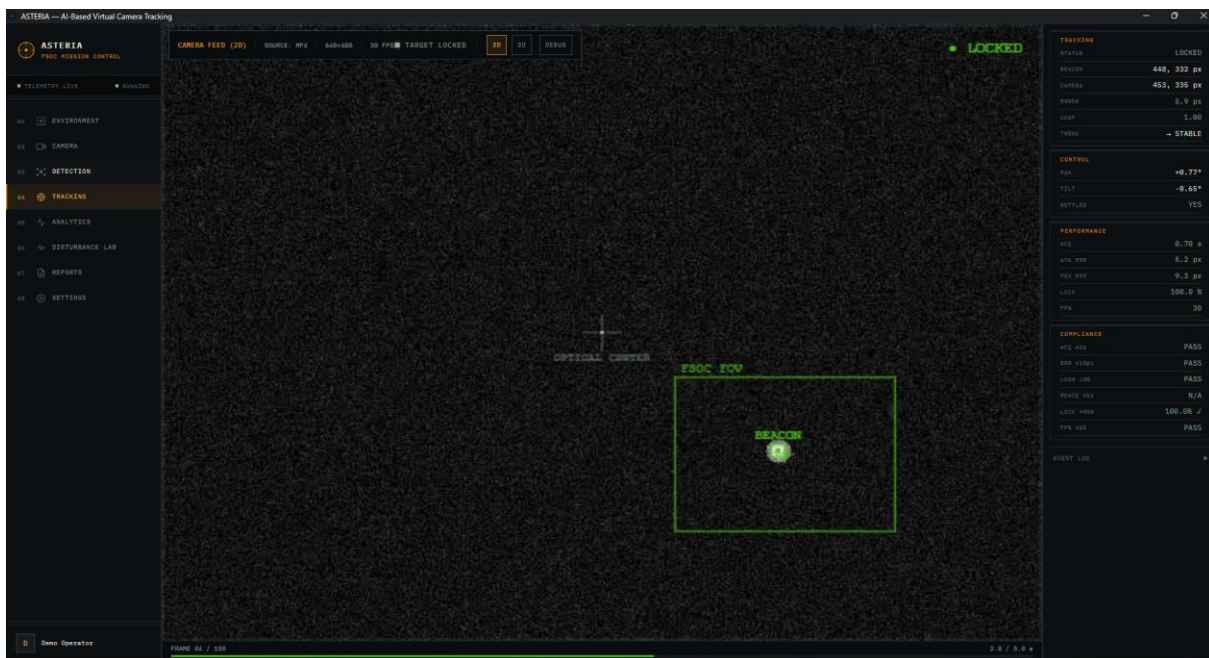


Figure 5. Uploaded-video tracking view: beacon, camera center, FOV and locked status are visible over a noisy frame.

6. AI Methods

What is actually AI-based in the main branch

ASTERIA contains both AI-assisted and non-AI tracking components. It is important to distinguish them in a technical submission.

Component	Status on main	Role
Classical CV detector	active	Beacon candidate detection and confidence scoring
Kalman filter	active	State estimation / prediction
PID controller	active	Pan/tilt feedback control
YOLO adapter	NOT complete	Extension point for learned detection
Gemini copilot	active when configured	Engineering Q&A and system guidance

YOLO verification

A **YOLO**Detector class exists, but its box-parsing **path** raises `NotImplementedError` and falls back to the classical detector. Therefore the main branch should not be described as having an operational **YOLO** tracker.

The detector interface is intentionally structured for a drop-in learned detector: a future **YOLO** implementation can return the same `DetectionResult` schema used by the classical detector. However, the repository does not include trained beacon weights or a labeled optical-beacon dataset, so **YOLO** should be described as an integration **path** rather than as an evaluated component of the current main-branch tracker.

Gemini copilot

The Node service uses **Gemini** 1.5 Flash with a domain prompt covering **FSOC**, atmospheric optics, **Kalman filtering**, **PID**, computer vision and performance analysis. It assists the operator but is not inside the deterministic real-time control loop.

The **Gemini** copilot is an advisory engineering layer. It can receive questions together with selected telemetry context and provide explanations or troubleshooting guidance, but its output is not allowed to directly command the real-time camera loop. This separation is important because generative responses are not deterministic enough to serve as a hard real-time controller.

Submission wording: “AI-assisted FSOC tracking platform with classical-CV tracking and a YOLO-ready vision interface.”

The intelligent portion is therefore best understood as a hybrid software pipeline: image-based perception, model-based estimation and feedback control operate deterministically, while generative AI is used as an advisory engineering interface. A future trained detector can be inserted behind the existing detector abstraction.

7. Test Methodology

Synthetic **closed-loop** validation and external video

The repository provides two complementary validation approaches. The headless Python suite exercises the numerical and **closed-loop** components under controlled, repeatable conditions, including trajectory generation, detector behavior, filtering, control direction, disturbances, target loss and recovery. A separate video-processing **path** runs the pipeline on uploaded MP4 files, providing a more realistic observation stream. Synthetic tests can use deterministic renderer seeds and retain ground truth for scoring, while external-video evaluation is limited to image-derived metrics unless annotations or another calibrated reference are supplied.

The synthetic suite is particularly valuable because it separates evaluation truth from observation: the renderer knows the exact target position, but the detector never receives that position as an input. This enables true synthetic localization error and **closed-loop** behavior to be measured without contaminating the perception **path**. The external-video **path** is complementary because it forces the pipeline to work from actual recorded image content; however, without frame-level labels it cannot provide independently verified **ground-truth** localization error.

The provided test organization follows a useful validation pyramid: low-level checks verify coordinate signs and equations; component tests exercise renderer, detector, **Kalman**, **PID** and disturbance behavior; acceptance cases combine those components. The supplied report records 39 passing Python tests, successful frontend and backend builds, and a successful Electron TypeScript build for the reviewed checkout. The suite also includes reset-independence checks so detector association memory from one video session cannot bias the first frames of another session.

Test group	Examples	Purpose
Trajectory	Linear, circular, figure-8, random	Motion robustness
Noise	Gaussian, salt-and-pepper, Poisson	Sensor robustness
Atmosphere	Haze, fog, rain, low light	Visibility degradation
Platform	Camera jitter, linear motion	Host disturbance
Recovery	Forced blackout / reacquisition	Loss handling
Repeatability	Seeded random initialization	Deterministic setup

Measured metrics

- Acquisition time
- Average / maximum pixel error and RMSE
- Target-loss percentage
- Lock retention
- Reacquisition time
- FPS and processing latency



Figure 6. Analytics dashboard showing error, FPS/latency, confidence and lock-state telemetry.

8. Performance Analysis

Observed run evidence and interpretation

The implementation is instrumented for quantitative performance reporting. RunMetrics accumulates **acquisition time**, **reacquisition** time, detection confidence, processing time, FPS, pixel/angular error, **lock retention**, target-loss events and state transitions. These values can be persisted with run metadata and exported through the reporting subsystem. For the supplied uploaded-video evidence, the generated report records a passing **PS169**-style evaluation for the shown run; these results should be treated as run-specific evidence rather than a universal performance guarantee.

A professional performance statement should distinguish throughput from tracking quality. FPS measures whether the pipeline can keep up with the intended cadence; **processing latency** measures computational headroom; **acquisition time** measures initial convergence; **tracking error** describes pointing residual; **lock retention** measures sustained stability; and **reacquisition** time measures recovery after loss.

Performance should be considered at both system and algorithm levels. FPS indicates throughput, latency indicates processing cost, and **tracking error**/retention describe control quality; none of these metrics alone is sufficient to characterize complete **PAT** behavior.

At the design target of 30 Hz, one complete frame has approximately 33.3 ms of wall-clock budget. Rendering and image degradation consume a portion of that budget, while connected-component extraction and candidate scoring are expected to be the dominant synthetic-mode CPU work. **Kalman** update/prediction and **PID** arithmetic are comparatively small fixed-size operations. **SQLite** persistence is deliberately kept off the critical event-loop **path**. For a stronger benchmark, the report should eventually publish p50/p90/p95/p99 frame times and per-stage timing rather than relying only on average FPS.

Metric	Observed run
Duration	6.94 s
Average FPS	30.0
Acquisition time	0.700 s
Average tracking error	5.34 px
Maximum tracking error	9.32 px
RMSE	5.45 px
Lock retention	100.0%
Detection confidence	98.9%
Processing latency	≈15.5 ms

The supplied report marks the **PS169**-style checks as pass for acquisition, **tracking error**, RMSE, **reacquisition** requirement (not required because no loss occurred), **lock retention**, processing speed and latency.



Figure 9. **PS169** specification compliance, lock-retention diagnostics, and error statistics from the generated ASTERIA performance report.

ASTERIA

AI-Based Virtual Camera Tracking — FSOC Coarse Alignment · PS169

SIMULATION PERFORMANCE REPORT

1. MISSION INFORMATION

Run ID	f32141c1-d6ef-49...
Scenario	VIDEO-05_circular_gaussian.mp4
Environment	VIDEO_INPUT
Start Time	2026-09-17 14:59:33
End Time	2026-09-17 14:59:40
Status	COMPLETED
Final State	LOCKED

2. TRACKING PERFORMANCE

Duration	6.94 s
Average FPS	30.0
Acquisition Time	0.700 s
Avg Re-acquisition Time	Not required (no loss)
Max Re-acquisition Time	—
Average Tracking Error	0.0558°
Maximum Tracking Error	1.0626°
Avg Tracking Error (px)	5.34 px
Max Tracking Error (px)	9.32 px
RMSE Tracking Error	5.45 px
Lock Retention	100.0%
Target Loss Count	—
Re-acquisition Count	—
Detection Confidence	98.9%
Avg Propagation Latency	15.50 ms

Figure 7. Generated performance report showing mission information and tracking-performance values.

Interpretation: this evidence applies to the shown run, not universally to all videos or conditions.

The values demonstrate that the implemented pipeline can satisfy the stated thresholds for the recorded case. Broader claims require repeated trials across trajectories, noise levels, atmospheric conditions and independent videos, preferably with confidence intervals.

9. Reporting and Data Persistence

Run history, exports and diagnostics

The reporting subsystem stores scenario and run metadata plus telemetry in **SQLite**. RunMetrics maintains bounded histories and scalar accumulators so the system can calculate performance statistics without repeatedly rescanning an entire session. Telemetry can be downsampled for persistence so database I/O does not become a bottleneck in the real-time loop, while video-mode processing can retain more complete telemetry for reproducible benchmark reports. The reporting layer supports CSV, json and PDF outputs, allowing both machine-readable analysis and presentation-ready evidence.

Capability	Implementation
Run history	SQLite run records with scenario, status and summary values
Telemetry	Time-series samples associated with a run
Exports	CSV, json and PDF
Diagnostics	Error statistics, retention and compliance checks
Frontend history	Bounded chart/event buffers

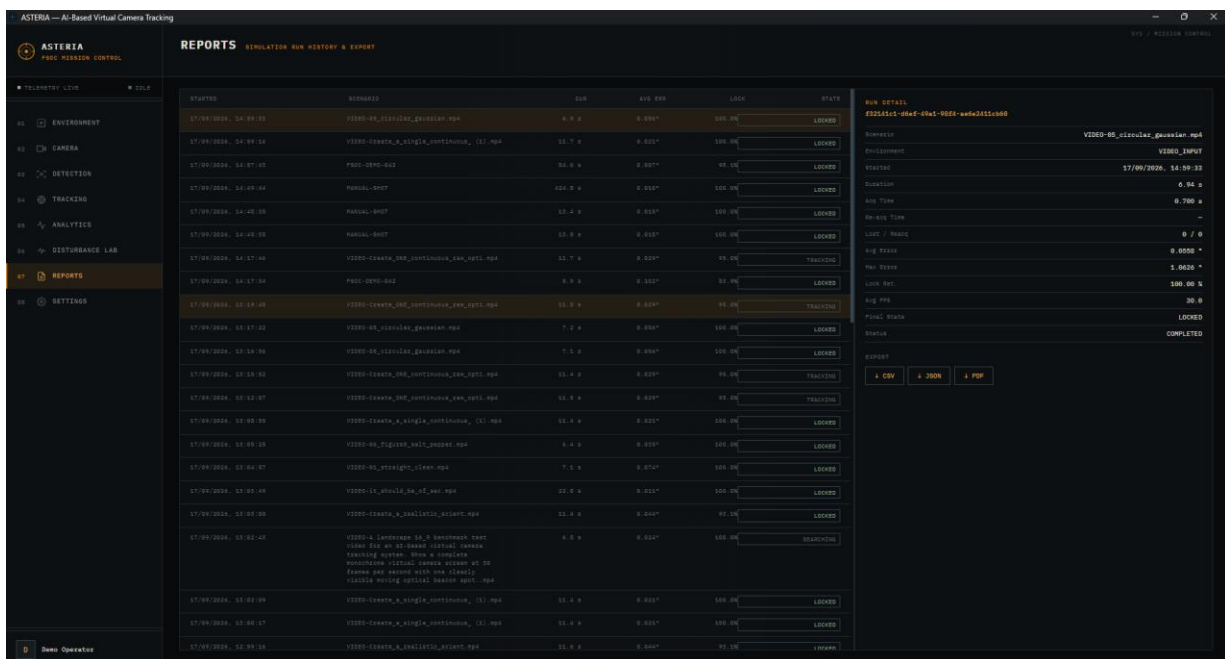


Figure 8. Reports page showing stored runs, final state, duration, error, lock retention and export options.

Data-quality note

For uploaded video, the system generally lacks an externally supplied **ground-truth** target coordinate. Its pixel residual is therefore best interpreted as a controller/aim residual, not as independently verified localization error.

For rigorous external-video evaluation, frame-level beacon annotations or a calibrated reference trajectory should be added. This would allow detector precision/recall and true localization error to be separated from controller residual error.

10. Robustness, Limitations and Technical Risks

What should be improved before a production-grade **PAT** system

Robustness strengths

- Finite-value sanitization protects control parameters from malformed inputs.
- Camera tilt is bounded to avoid projection singularities.
- Detector continuity reduces frame-to-frame candidate jumping.
- Lock hysteresis reduces false unlocks.
- Dropout coast and explicit **reacquisition** handle temporary target loss.

Current limitations

- Classical detection can confuse bright non-beacon structures.

The main technical transfer risk is the gap between the renderer's controlled beacon appearance and real optical imagery. Lens flare, bloom, compression, background stars, motion blur and real atmospheric scattering can alter the feature distribution seen by the detector. The **Kalman** model is also intentionally simple: constant velocity is effective for short prediction horizons but is less representative of abrupt acceleration or strongly nonlinear motion.

- **YOLO** integration is incomplete and falls back to classical CV.
- External-video residuals are not **ground-truth** localization error.
- Benchmark scripts contain environment-specific video **paths**.
- Production deployment should **restrict** cors and add authentication/rate limiting.

A practical next step is to add a repeatable benchmark harness that sweeps trajectories, FOV values, **PID** gains, detector configurations and disturbance strengths over fixed seeds. Results should report acquisition success, median and tail **acquisition time**, error percentiles, **lock retention**, **reacquisition** distributions and frame-time percentiles. Longer term, a labeled beacon dataset and **hardware-in-the-loop** setup would allow the learned detector, camera geometry and controller to be evaluated against representative physical behavior.

11. Future Improvements and Conclusion

Roadmap from simulator to stronger validation

Area	Improvement
Vision accuracy	Improve beacon detection for glare, blur, low light, haze and complex backgrounds.
AI detection	Complete and evaluate the YOLO-based detector using domain-specific optical-beacon data.
Tracking stability	Add acceleration-aware prediction and stronger target association during rapid motion.
Control tuning	Introduce automated PID tuning and actuator-delay modelling for faster, smoother response.
Evaluation	Use frame-level ground-truth annotations to measure detection accuracy and true localization error.
Real-world validation	Add camera/gimbal calibration and hardware-in-the-loop testing with representative optical conditions.
Benchmarking	Automate multi-scenario testing with fixed seeds, regression baselines and detailed performance statistics.
Deployment	Strengthen authentication, CORS restrictions, upload validation, HTTPS and secret management.

Final technical conclusion

ASTERIA's main branch provides a real **closed-loop** software **PAT** environment rather than a UI-only mock. The core **path**—image detection, **Kalman** estimation, **PID** control, **tracking state** management, disturbance injection, telemetry and performance reporting—is implemented and instrumented.

The supplied ASTERIA interface screenshots further demonstrate how these internals are exposed operationally: the 3-D view shows **spatial** entities and lock state, the 2-D view shows the FOV and beacon relative to the optical center, the camera-control page exposes **pan/tilt** and **PID** behavior, the analytics page plots historical telemetry, and the reports page preserves run-level evidence.

For an academic or hackathon submission, the most defensible description is an **FSOC** coarse-**PAT** simulation and validation platform with AI-assisted tooling. Numerical performance claims should always be tied to a specific executed run, and **YOLO** should be presented as a future/unfinished detector integration unless separately completed and benchmarked.

11. Conclusion

ASTERIA demonstrates a complete software-based environment for coarse **Pointing, Acquisition and Tracking (PAT)** in a **Free-Space Optical Communication** scenario. The implementation connects target-motion simulation, camera projection, **beacon detection**, **Kalman** state estimation, **PID**-based **pan/tilt** control, tracking-state management, telemetry, analytics and run reporting into a single **closed-loop** workflow. This makes the platform suitable for demonstrating how a virtual optical terminal can acquire and maintain alignment with a moving beacon while exposing the internal measurements and control decisions to the operator.

The main-branch implementation also provides useful experimental controls through multiple trajectories and disturbance conditions, together with quantitative metrics such as **acquisition time**, **tracking error**, RMSE, **lock retention**, processing speed and latency. The supplied performance evidence shows that the demonstrated run achieved the stated **PS169**-style thresholds, including **0.700 s** acquisition, **5.3 px** average **tracking error**, **5.5 px** RMSE, 100% **lock retention**, **30 FPS** and approximately **15.5 ms processing latency**. These values should be interpreted as results for the documented run rather than as universal performance guarantees.

A key technical distinction is that the current operational tracking **path** is based on **classical computer vision**, **Kalman filtering** and **PID** control. The repository contains a **YOLO** integration point, but the learned detector is not a completed production **path** in the main branch. The **Gemini** engineering copilot is advisory and is separated from the deterministic **real-time tracking loop**. Consequently, the system is best characterized as an AI-assisted **FSOC** tracking and validation platform with an extensible learned-detection interface.

For the next development stage, the strongest improvements are **ground-truth** annotated video evaluation, completion and benchmarking of the **YOLO** detector, camera and gimbal calibration, **hardware-in-the-loop** testing, broader long-duration stress testing, and stronger deployment controls. With these additions, ASTERIA can progress from a capable simulation and demonstration platform toward a more rigorous research and hardware-validation framework for optical tracking systems.